

Industrial Defect Inspection & Surface Crack Detection via Custom Linear Filtering and Edge Processing

Dania Waseem

23i-2622

BS Data Science

Department of Artificial Intelligence and Data Science

National University of Computer and Emerging Sciences

CS4059 Fundamentals of Computer Vision, Fall 2026

Introduction

Factories that make things like circuit boards or concrete panels need a way to automatically check surfaces for problems such as broken traces on a PCB or thin cracks in concrete. Cameras capture these surfaces, but the images are never perfectly clean. There is sensor noise, blur, and uneven lighting to deal with. Before a computer can find a defect, the image usually needs to be cleaned up first and its edges need to be found clearly.

This report explains an implementation of that whole process, built completely from scratch using only NumPy. No built in OpenCV filtering functions such as `filter2D`, `GaussianBlur`, or `Canny` were used. The same code was tested on two very different datasets. One is the PCB Defect dataset, which has sharp, high-contrast circuit boards. The other is the Surface Crack Detection dataset, which has softer, lower-contrast concrete images. Testing on both means the results are not just true for one type of image.

Task 1: Convolution From Scratch

2D Convolution

The function `conv2d` slides a kernel over an image and, at every position, multiplies the kernel with the patch of image underneath it and adds up the result. This is technically called cross correlation since the kernel is not flipped. The assignment allows this because it is what almost every modern computer vision library actually does anyway.

The function supports two padding modes:

- `same`: the output image stays the same size as the input by adding zero padding around the borders.
- `valid`: no padding is added, so the output naturally shrinks because the kernel cannot fully fit near the edges.

It also supports a configurable stride, which controls how many pixels the kernel jumps between each computation. The output size follows the standard formula $W_{out} = \text{floor}((W_{in} - F + 2P) / S) + 1$,

and $H_{out} = \text{floor}((H_{in} - F + 2P) / S) + 1$, where F is the kernel size, P is the padding, and S is the stride.

To check this worked correctly, a simple 3x3 averaging kernel was applied to sample images from both datasets. The output shapes were printed and compared against the formula above under same, valid, and stride=2 settings. All of them matched.

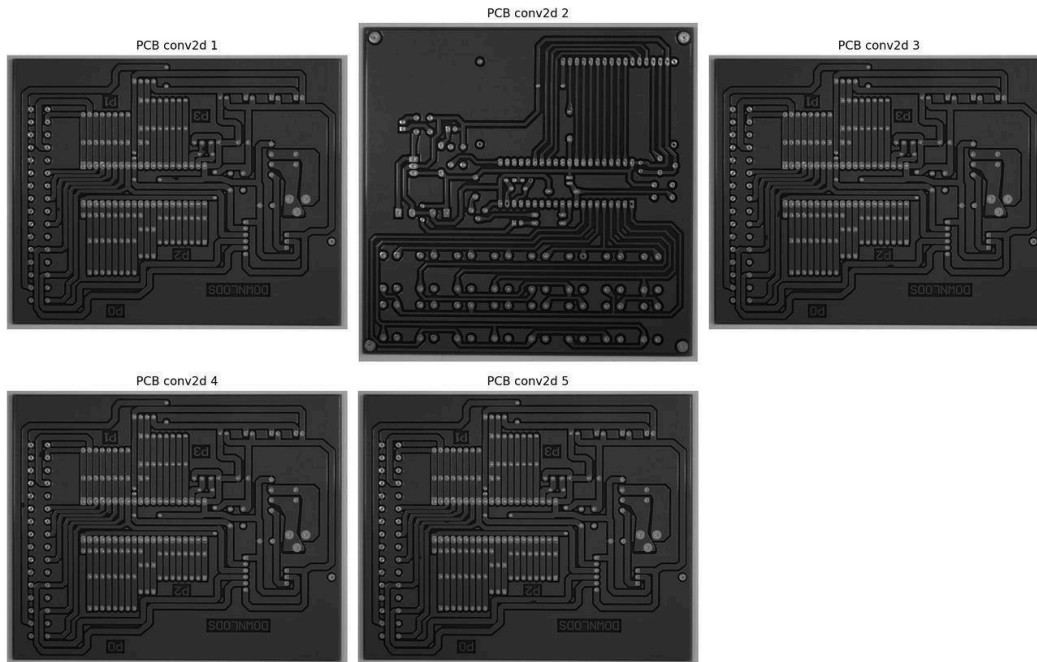


Figure 1. conv2d output (3x3 average kernel) on PCB samples.

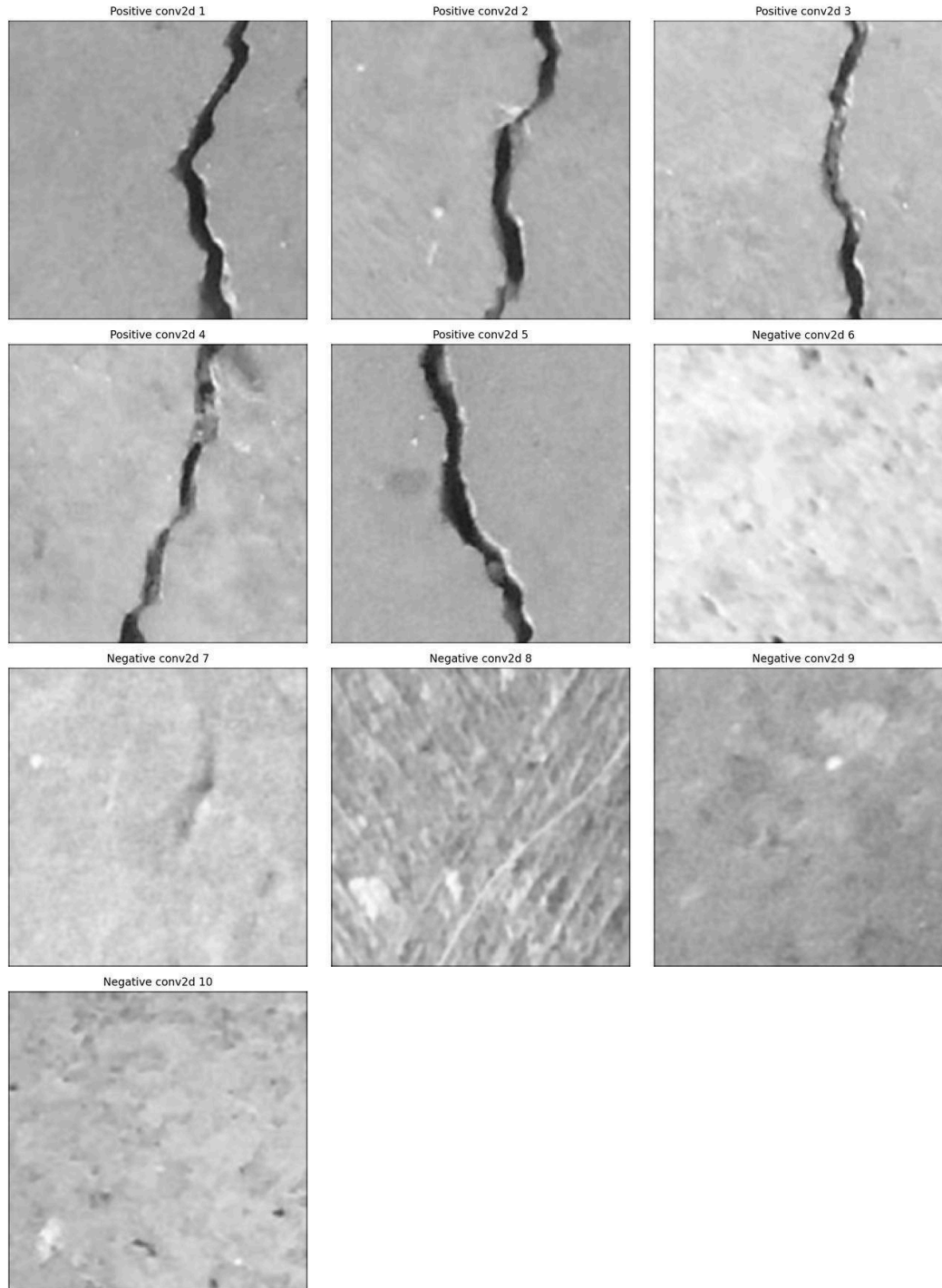


Figure 2. conv2d output on Surface Crack samples. The same kernel produces the same kind of smoothing on both datasets, which shows the function is not tied to one type of image.

3D Convolution (Multiple Channels)

Real world images are usually color images with three channels: red, green, and blue. Not grayscale. conv3d extends the same idea to color images. It takes an image of shape (H, W, C) and a stack of K

kernels, each of shape (k_h, k_w, C) . For every kernel in the stack, it multiplies the kernel with the matching patch across all channels, adds everything up into a single number, then stacks the K results together to form an output volume of shape (H_{out}, W_{out}, K) .

To test this, three kernels were built. One averages all three color channels together. One only looks at the red channel. One only looks at the green channel. Running these on both datasets produced output volumes with the correct shape $(H_{out}, W_{out}, 3)$ every time. Each channel-specific kernel correctly picked out only its assigned channel information, which confirms the per-channel logic works as intended.

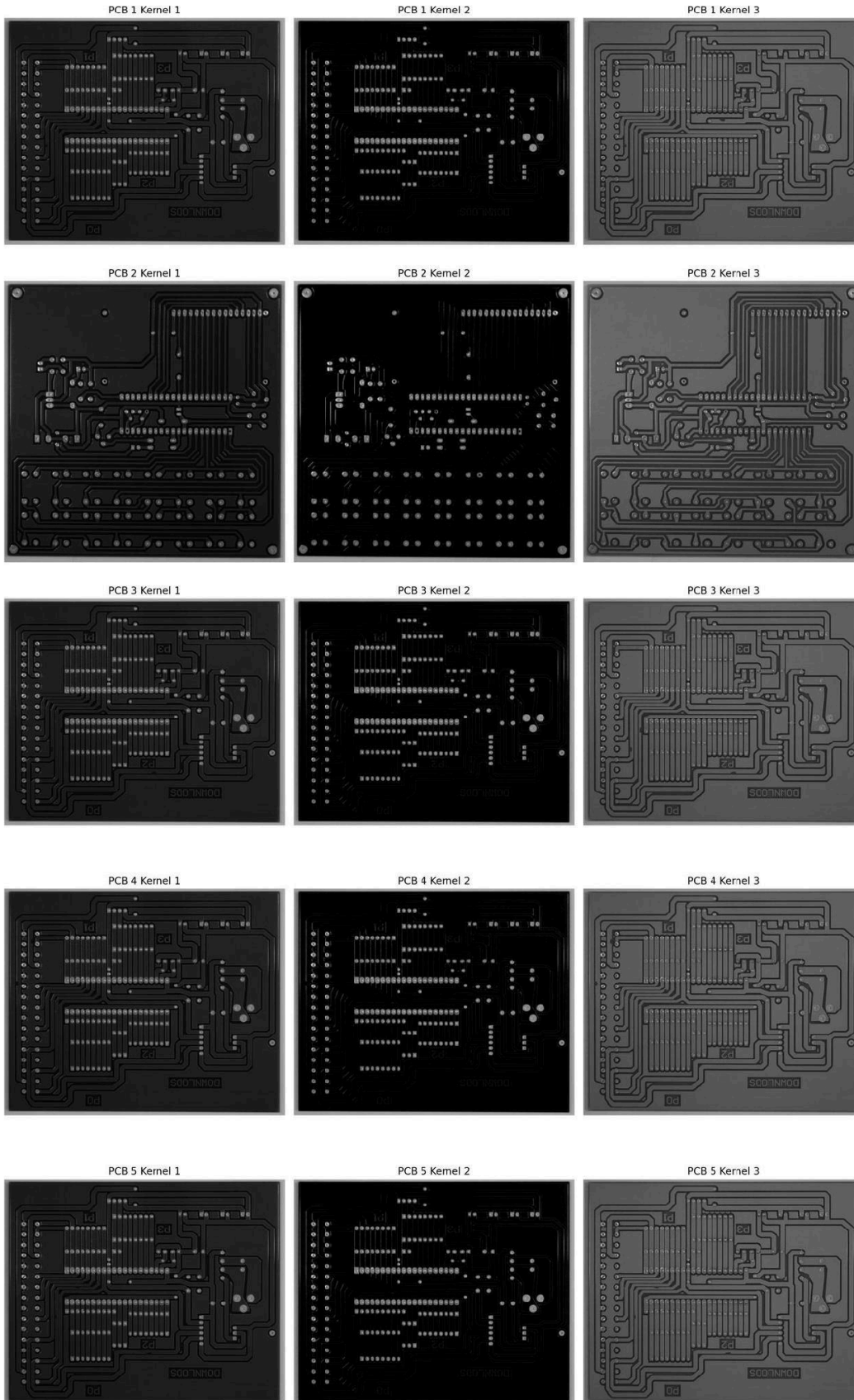
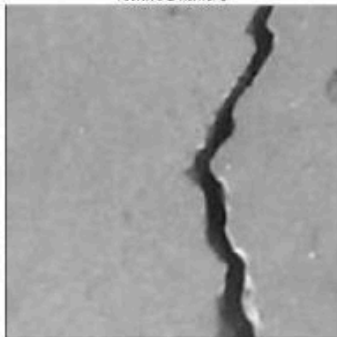
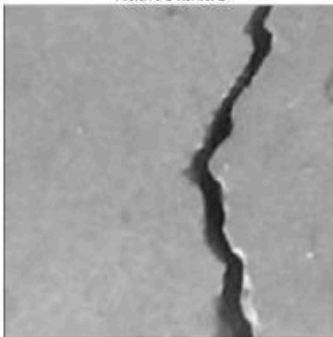


Figure 3. conv3d output on PCB samples using the three test kernels: RGB average, red only, and green only.

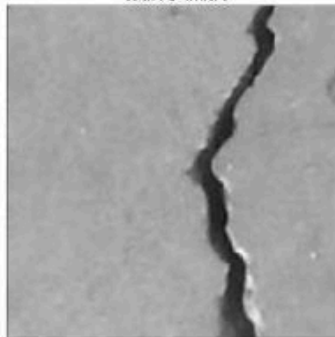
Positive 1 Kernel 1



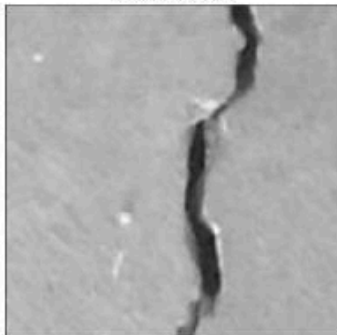
Positive 1 Kernel 2



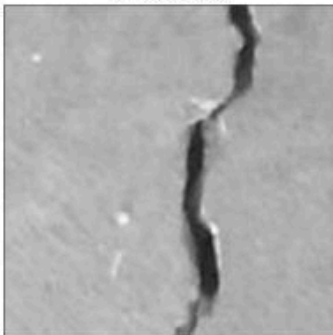
Positive 1 Kernel 3



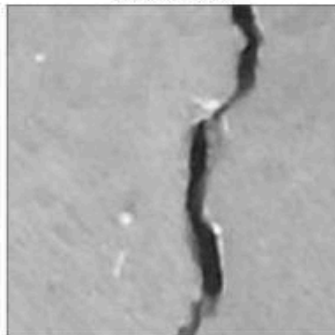
Positive 2 Kernel 1



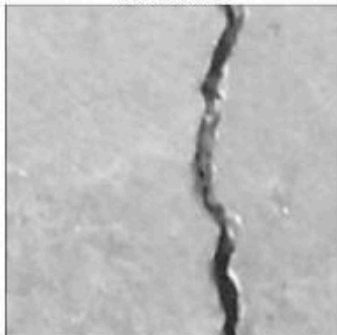
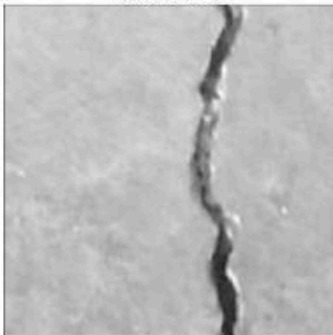
Positive 2 Kernel 2



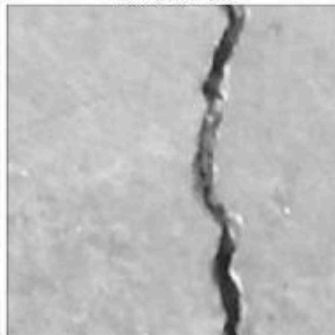
Positive 2 Kernel 3



Positive 3 Kernel 1

Positive Ξ Kernel Ξ 

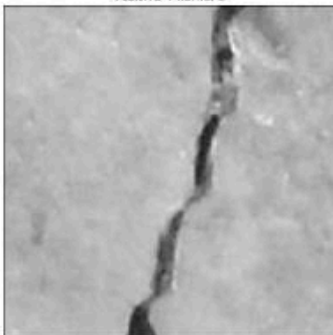
Positive 3 Kernel 3



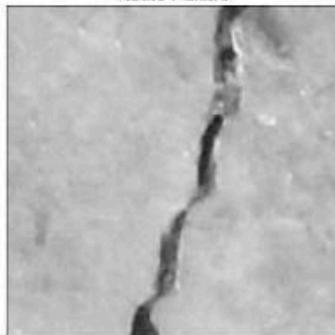
Positive 4 Kernel 1



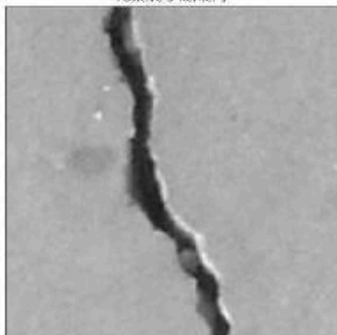
Positive 4 Kernel 2



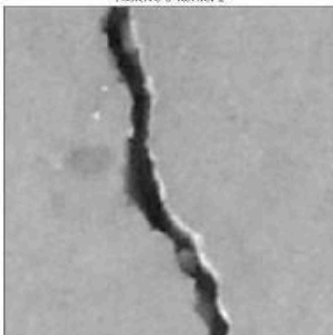
Positive 4 Kernel 3



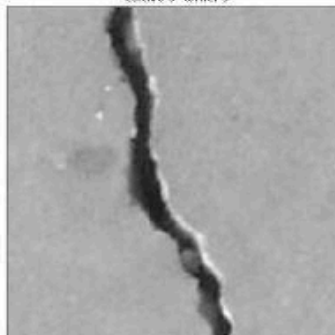
Positive 5 Kernel 1



Positive 5 Kernel 2



Positive 5 Kernel 3



Negative @ Kernel 1



Negative 6 Kernel 2



Negative 6 Kernel 3



Figure 4. conv3d output on Surface Crack samples. The channel-selective kernels correctly isolate their assigned channel on both datasets.

Task 2: Smoothing, Sharpening, and Pooling

Gaussian Kernel

A Gaussian kernel is a small grid of numbers that gives more weight to pixels close to the center and less weight to pixels further away. This is what makes it useful for blurring or smoothing an image. It is built using the Gaussian formula generated over a coordinate grid using `np.meshgrid`. After generating it, the kernel is re normalized so all its values add up to exactly 1. This matters because blurring an image with this kernel will not make the picture darker or brighter overall. It only makes it smoother.

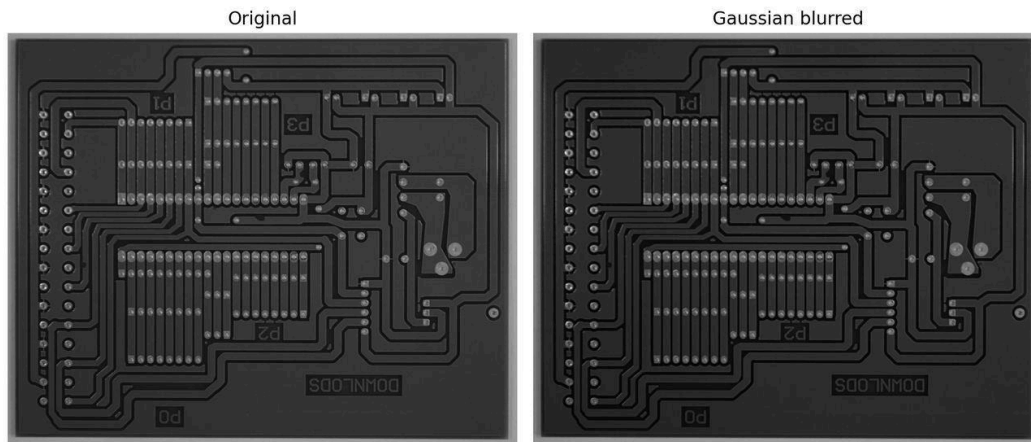


Figure 5. Gaussian blur, $\sigma = 1.4$, applied using `conv2d`.

Unsharp Masking

Unsharp masking is a simple trick to make an image look sharper. First the image is blurred slightly. Then the blurred version is subtracted from the original. What is left over (the "detail") is added back on top of the original image, boosted by a strength factor α : $F_{\text{sharp}} = F + \alpha * (F - F * G_{\text{sigma}})$. The final result is clipped to the valid pixel range $[0, 255]$ so no overflow artifacts appear.

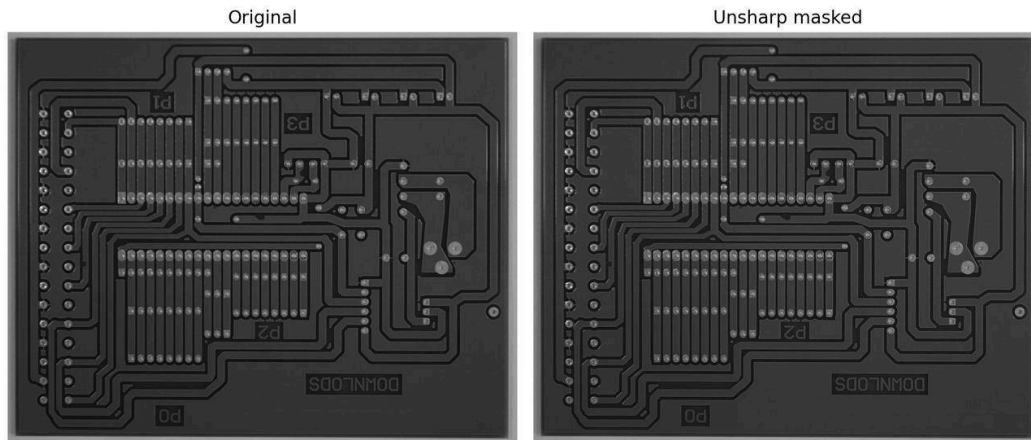


Figure 6. Unsharp masking result, $\sigma = 1.5$, $\alpha = 1.2$.

Pooling

Pooling shrinks an image (or feature map) by summarizing small blocks of pixels into a single number. Two modes were implemented. Max pooling keeps only the largest value in each block. Average pooling takes the mean of each block.

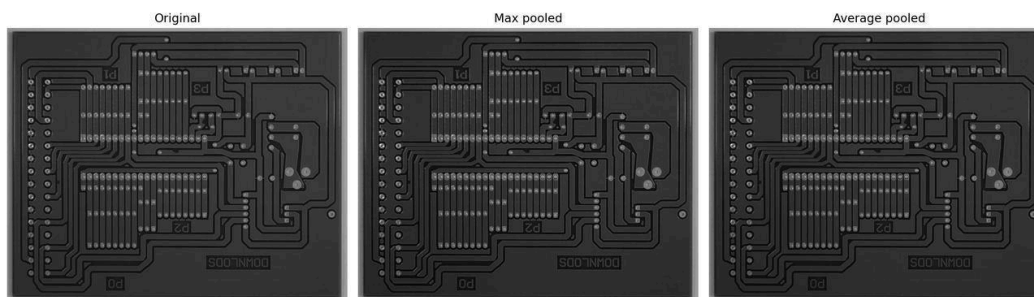


Figure 7. Max pooling vs. average pooling, 2×2 blocks, stride 2.

Theoretical reflection. The assignment asks for the gradient of max pooling for the input block $[[8, 7], [12, 9]]$. The largest value in this block is 12, in the bottom left position. So the gradient matrix is $[[0, 0], [1, 0]]$. In plain words, max pooling only "chose" one number out of the four (the 12) and threw the other three away. So if we were training a network and gradients flowed backward through this operation, all of the gradient would go straight to wherever that maximum value came from. The other three positions would get zero gradient because they had no effect on the output in the first place.

Task 3: Canny Edge Detection

The Canny edge detector is built here in four separate stages, exactly as described in the assignment.

Stage 1: Gradients. The image is first smoothed with the Gaussian kernel from Task 2. Then two Sobel kernels are convolved with it: S_x for horizontal change and S_y for vertical change. From these two results we compute the gradient magnitude $M = \sqrt{S_x^2 + S_y^2}$, which tells us how strong the edge is, and the gradient direction $\theta = \text{atan2}(S_y, S_x)$, which tells us which way the edge is pointing.

Stage 2: Non Maximum Suppression (NMS). The direction θ is rounded into one of four buckets: 0, 45, 90, or 135 degrees. For every pixel, its magnitude is compared against its two neighbours in that direction. If the pixel is not strictly bigger than both neighbours it gets zeroed out. This is what turns a thick blurry band of "maybe an edge here" pixels into a single clean line exactly one pixel wide.

Stage 3 and 4: Double Thresholding and Hysteresis. Every remaining pixel is placed into one of three groups. Strong pixels are definitely an edge. Weak pixels might be an edge or might just be noise. Non edge pixels get thrown away entirely. Weak pixels are only kept if they are directly or indirectly touching a Strong pixel, checked using 8-connectivity so that diagonal neighbours count too. This is implemented here with a queue based search (BFS) so the check runs efficiently even on large images.

A note on choosing the threshold values. At first, fixed numbers were used for the Strong and Weak cutoff. This caused problems. Values that worked fine for the high contrast PCB images produced almost completely empty (black) edge maps on the lower contrast Surface Crack images, because their gradient values are naturally much smaller. To fix this properly, the thresholds are instead computed automatically for each image using Otsu's method. It looks at the histogram of gradient values in that specific image and finds the cutoff point that best separates edge pixels from background pixels. This means the same code works well on both datasets without needing different hand picked numbers for each one.

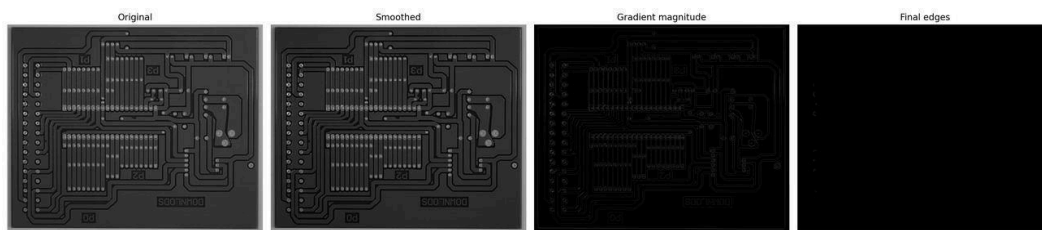


Figure 8. Full Canny pipeline on a PCB image: original, smoothed, gradient magnitude, and final edges. Thresholds picked automatically with Otsu's method.

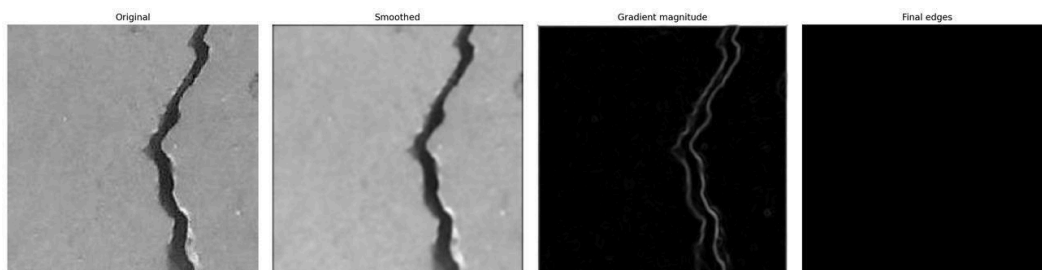


Figure 9. Full Canny pipeline on a Surface Crack image.

Task 4: Ablation Study

An ablation study means removing or changing one part of the pipeline at a time to see exactly what that part contributes.

Ablation 1: Does Smoothing Actually Help With Noise?

Random Gaussian noise was added to a test image at multiple noise levels, and gradients were computed both with and without the Gaussian pre smoothing step. The idea is to see whether smoothing before computing gradients actually removes the noise or whether it was an unnecessary step.

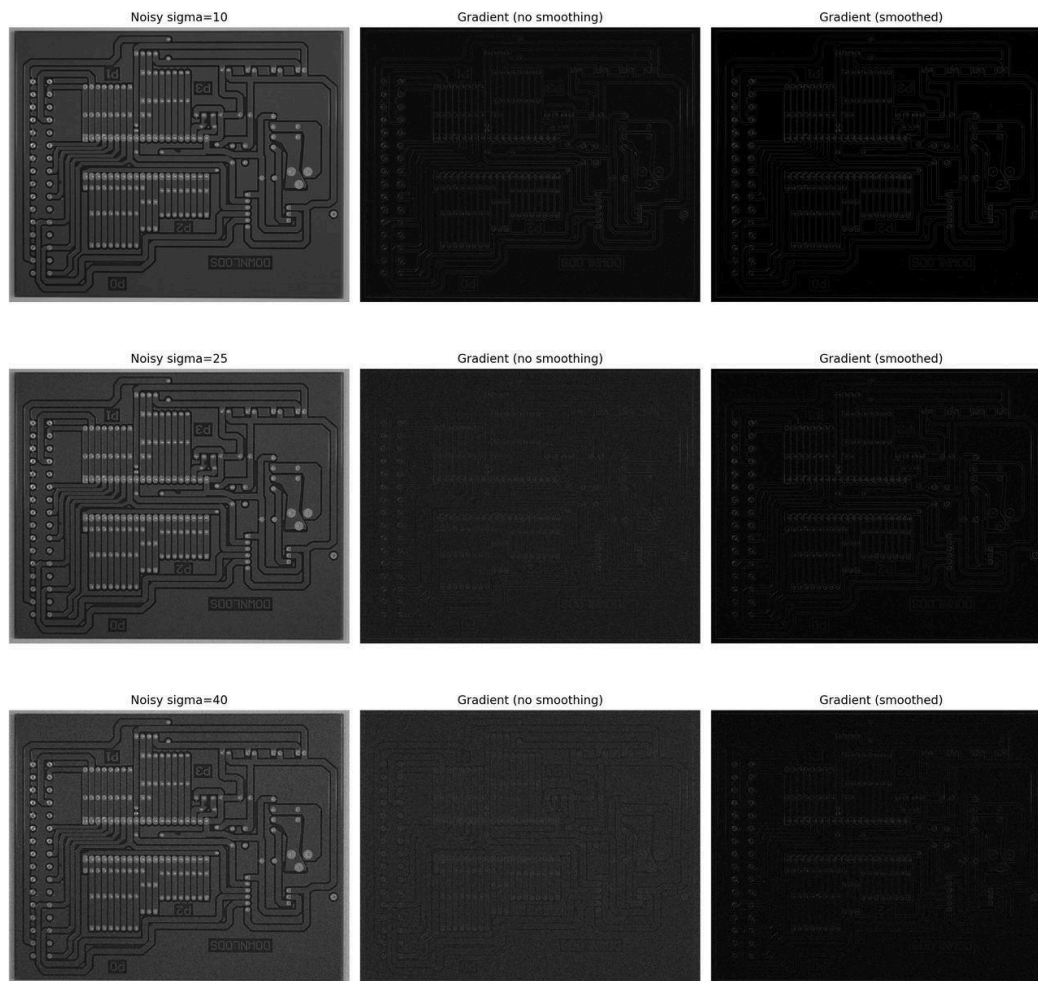


Figure 10. Noise vs. smoothing ablation.

Without smoothing, the noise itself creates lots of small fake gradients scattered all over the image. This would confuse the edge detector into thinking there are edges everywhere. After Gaussian smoothing, most of these fake gradients disappear while the real structural edges, such as PCB traces or crack lines, stay visible. This confirms that the smoothing step in the pipeline is genuinely necessary and not just extra work.

Ablation 2: What Does NMS Actually Do?

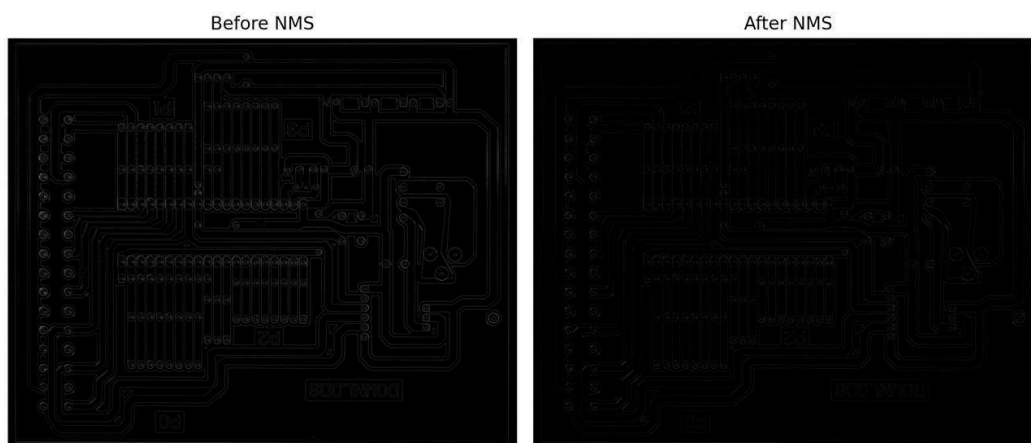


Figure 11. Gradient magnitude before and after NMS.

Before NMS, the edges in the gradient magnitude image look like thick, fuzzy bands several pixels wide. This happens because many nearby pixels all have a high gradient value around the same real edge. After NMS, these bands are thinned down into single pixel wide lines, since only the strongest pixel along the gradient direction at each spot survives.

Ablation 3: Hysteresis vs. a Single Threshold

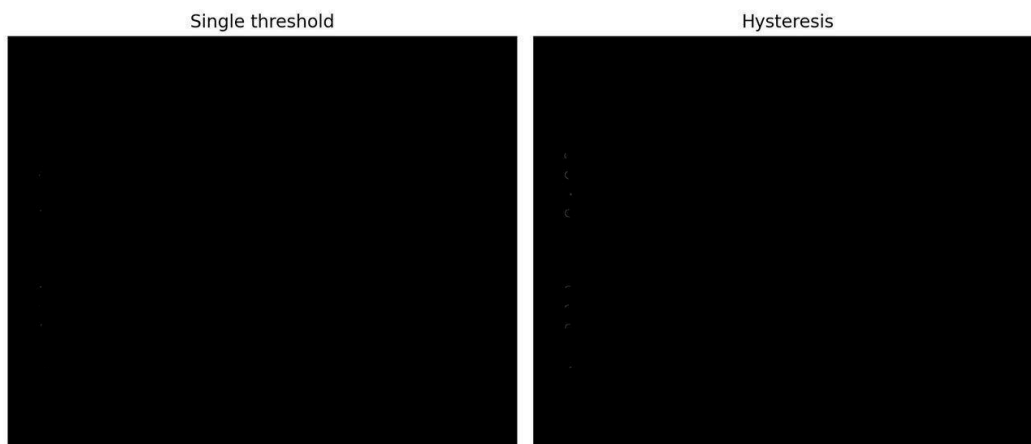


Figure 12. A single fixed threshold vs. double threshold hysteresis.

Using only one global threshold forces a trade off. Set it too high and faint but real crack lines get broken into disconnected pieces or disappear completely. Set it too low and background noise starts sneaking in as fake edges. Hysteresis avoids this trade off by keeping weak pixels only when they are connected to a strong pixel. Faint but genuine crack segments stay connected as one continuous line while isolated noise pixels, which are not connected to anything strong, get dropped.

Task 5: Scale Space, Sensitivity, and Failure Cases

Scale Space Behaviour

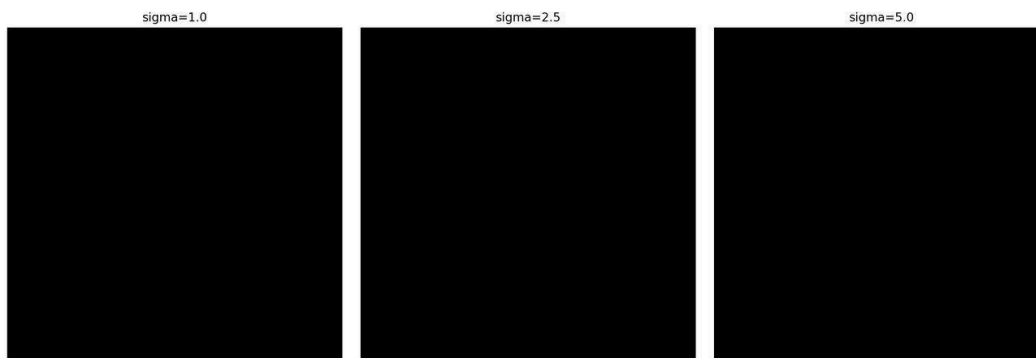


Figure 13. Edges at $\sigma = 1.0$, 2.5 , and 5.0 .

As σ , the amount of Gaussian smoothing, increases, small details in the image get blended into larger, coarser shapes. This is what the theory calls Witkin's scale space causality. As you smooth more, edges can only merge together, get weaker, or shift slightly. The smoothing process can never create a brand new edge that was not already there at a finer scale. In other words, blurring cannot invent detail. It can only remove it.

Parameter Sensitivity

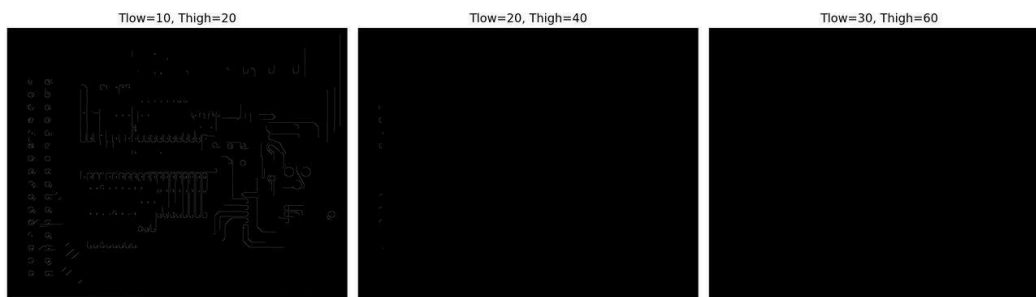


Figure 14. Threshold sensitivity sweep.

Instead of testing arbitrary fixed threshold numbers, this sweep uses Otsu's method to pick Thigh automatically for each image, then varies only the ratio $T_{low} = r * T_{high}$ for r in $\{0.3, 0.5, 0.7\}$. This choice came directly from an earlier problem encountered while working on this assignment. Using the same fixed threshold numbers on both datasets did not work at all. Values tuned for PCB sharp, high contrast traces produced almost completely empty edge maps on the lower contrast Surface Crack images. The opposite problem also showed up: thresholds that were far too small for PCB meant every single pixel passed the threshold no matter how it was changed, so the edge map never changed at all across the whole sweep. Even picking threshold values based on a fixed percentile of the image brightness values did not fully solve this, since a single unusually bright pixel, for example an artifact or a label in the corner of an image, could distort the chosen percentile far above where

the real edges actually were. Otsu's method fixes this because it looks at the actual shape of each image's own gradient histogram and finds the natural split point between background and edge pixels rather than assuming a fixed cutoff. With this approach, increasing r from 0.3 to 0.7 causes edges to be dropped gradually and predictably, instead of jumping straight from "everything is an edge" to "nothing is an edge."

Failure Cases

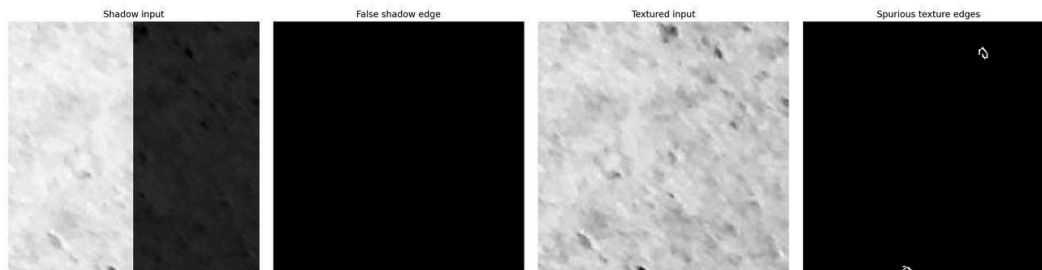


Figure 15. Shadow and texture failure cases.

Shadow contours. A sharp shadow boundary in an image creates a big jump in brightness, exactly the same way a real physical edge would. Canny only looks at how much the brightness changes. It has no way of knowing whether that change came from an actual crack or defect, or just from a light source casting a shadow. So it marks a strong "edge" along the shadow line even though nothing is physically wrong with the surface there. This same failure shows up on both PCB and Surface Crack images because it is a limitation of gradient based edge detection in general, not something specific to one dataset.

Highly textured surfaces. Rough or grainy surfaces, like textured concrete, naturally have lots of small brightness changes everywhere even where there is no real defect. These small gradients can pass the threshold and flood the output with many tiny fake edges, making it hard to tell which lines are the actual crack and which are just surface texture. Increasing the smoothing amount (sigma) reduces this noise but also blurs out genuinely thin cracks at the same time. There is no single setting that perfectly solves both problems at once, which matches what was already seen in the threshold sensitivity experiment above.

Conclusion

Every part of the pipeline required by the assignment was implemented using only NumPy: conv2d, conv3d, the Gaussian kernel, unsharp masking, pooling, and the full four stage Canny edge detector. All of it was tested on both the PCB Defect dataset and the Surface Crack Detection dataset. The convolution, smoothing, NMS, and scale space behaviour all worked consistently across both datasets, which confirms these parts of the implementation are correct and not accidentally tuned to just one type of image. The one part that genuinely needed extra care was choosing the edge thresholds. Fixed numbers did not generalize between a high contrast dataset like PCB and a low contrast one like Surface Crack, so an automatic, image specific approach using Otsu's method was

used instead. This produced sensible, comparable edge maps on both datasets without any manual re-tuning.